

A COMPANION GUIDE TO YOUR OWN CODE

NeoJob

Code Walkthrough & Reference Guide

Every page. Every function. Every API call — explained in plain English, in the order a visitor actually experiences the site.

Written so you can answer any question about how NeoJob actually works, line by line, during your defense.

Companion to the NeoJob project by Alaa Eddine Bendahane

How to Use This Guide

This document has one job: after reading it, you should be able to open any file in the NeoJob project, point at any function, and explain — in your own words — what it does, why it exists, and what would break if it were removed.

We start from Part 1 with the concepts that make everything else make sense: what an API actually is, why the code is full of the word “await”, and how a browser and a server talk to each other at all. If you already feel solid on those ideas, skim Part 1 and jump ahead — but if “async/await” has ever confused you, read Part 1 slowly. Every later part leans on it.

From Part 2 onward, the guide follows one continuous, successive path rather than jumping around the file tree at random. The entry point we chose is `index.html` — the page that loads the instant someone types `neojob.ma` into a browser, exactly the way a real visitor would experience the site. From there, we follow the same journey a curious visitor would actually take: browse the homepage, search for a job, read a job's details, check out the company, maybe use the salary simulator, then decide to sign up. Once we cross into “logged in” territory, we follow the candidat journey start to finish, then the recruteur journey, then the admin back-office — each in the order a real user of that role would click through the site.

Before any of that, Part 2 covers the handful of shared files that get loaded on almost every single page (`db.js`, `auth.js`, `components.js`, `notifications.js`, `main.js`) and Part 3 covers the PHP API those files talk to. We explain those once, thoroughly, up front — so that later, when a page's walkthrough says “this calls `Jobs.getPublished()`”, you can flip back and already know exactly what that does instead of re-reading the same explanation thirty times.

HOW TO READ THE CODE BOXES Every code snippet in this guide is copied verbatim from the actual project files — nothing here is simplified pseudocode. Where a snippet is trimmed for length, it's marked accordingly, and the file path is always given so you can open the real file and follow along.

Table of Contents

(This updates automatically in Microsoft Word: right-click it → “Update Field”, or press Ctrl+A then F9.)

How to Use This Guide	2
Table of Contents	3
Part 1 — The Concepts You Need First	6
1.1 How a Website Like NeoJob Actually Works.....	6
1.2 What HTML, CSS, and JavaScript Each Do.....	6
1.3 What PHP Does	7
1.4 Databases, SQL, and PDO — the Short Version	7
1.5 What Is an API — and Why Does NeoJob Have One?	8
1.6 JSON — the Language Between Front-End and Back-End	9
1.7 Synchronous vs. Asynchronous — Why “await” Exists	9
1.8 A Careful, Real-Code Walkthrough of async/await	10
1.9 Sessions and Cookies — How the Site Remembers You're Logged In	12
1.10 Putting It All Together — One Full Request, Start to Finish	13
Part 2 — The Shared Toolkit.....	15
2.1 db.js — Talking to the API	15
2.2 auth.js — Login, Sessions, and Guarding Pages	18
2.3 components.js — Header, Sidebar, and Reusable Cards	20
2.4 notifications.js — The Notification Bell	21
2.5 main.js — NeoUI, NeoTheme, and the Command Palette.....	22
Feedback to the user	22
Formatting.....	22
Avatars, ratings, and files	23
The command palette (Ctrl+K)	23
2.6 charts.js — Dashboard Charts	23

2.7 quiz-data.js — the Skill Verification Question Bank	24
Part 3 — The Backend: Inside the API	25
3.1 The Three Shared Files	25
3.2 auth.php — Login and Account Creation	26
3.3 The Join-Table Pattern — One Idea That Repeats Constantly	27
3.4 offres.php — Job Postings	28
3.5 entreprises.php — Companies	29
3.6 candidats.php — Candidate Profiles	30
3.7 candidatures.php — Job Applications	30
3.8 entretiens.php — Interview Scheduling	31
3.9 favoris.php, paiements.php, alertes.php, messages.php — the Smaller Files	32
3.10 avis.php — Company Reviews (the best-secured file)	32
3.11 categories.php and competences.php — Reference Lists	33
3.12 utilisateurs.php — Accounts (the most nuanced auth logic)	33
3.13 upload.php — File Uploads (the most carefully secured file)	34
Part 4 — Page by Page, In the Order a Real User Sees Them	36
4.1 Public Pages — Before You Log In	36
index.html — the Homepage	36
offres.html — Search and Filter Jobs	37
offre-detail.html — One Job's Full Page	37
entreprise.html — A Company's Public Page	38
simulateur-salaire.html — Salary Estimator	39
tarifs.html — Pricing	39
contact.html, a-propos.html, faq.html, 404.html — the Static Pages	39
register.html and login.html — Creating and Entering an Account	40
candidat-profil.html (project root) — a Candidate's Public CV	40
4.2 The Candidat Journey	40
candidat/dashboard.html — the Candidat's Home Screen	41

candidat/profil.html — Edit My Profile (+ the Skill Quiz).....	41
candidat/candidatures.html — Tracking My Applications.....	42
candidat/favoris.html — Saved Jobs.....	43
candidat/alertes.html — My Saved Search Alerts	43
parametres.html — Account Settings (Shared by Every Role).....	43
4.3 The Recruteur Journey	44
recruteur/dashboard.html — the Recruiter's Home Screen.....	44
recruteur/poster-offre.html — Publish (or Edit) a Job.....	44
recruteur/mes-offres.html — My Job Postings.....	44
recruteur/candidatures-recues.html — the Kanban Pipeline (Deep Dive).....	45
recruteur/rechercher-candidats.html — the CVthèque	47
recruteur/profil.html — Edit My Company.....	48
recruteur/paiement.html — Simulated Checkout	48
A note on the recruteur journey overall	48
4.4 The Admin Journey.....	49
admin/dashboard.html — Platform Overview.....	49
admin/moderation.html — Approve or Reject Job Postings	49
admin/utilisateurs.html — Manage Every Account.....	49
admin/entreprises.html — Grant or Revoke the Verified Badge.....	50
admin/avis.html — Moderate Company Reviews	50
admin/categories.html and admin/competences.html — the Reference-List Editors.....	50
Part 5 — Things Worth Knowing for Q&A.....	51
5.1 Recurring Patterns Cheat-Sheet.....	51
5.2 Known Bugs and Gaps — Good, Honest Answers if Asked “What Would You Improve?”	52
5.3 Quick Glossary	53
Closing Note	55

Part 1 — The Concepts You Need First

This part explains, in plain language, every idea the rest of this guide leans on: client vs. server, what an API is, what JSON is, and — the part most people find hardest — what “async” and “await” actually mean and why NeoJob's code is full of them. Nothing here is specific to NeoJob yet; these are general web-development ideas. Part 2 onward applies them to the actual project.

1.1 How a Website Like NeoJob Actually Works

Two separate computers are involved every time you use NeoJob, and it's important to keep them mentally separate:

- The client — your browser (Chrome, Edge, etc.), running on your own laptop or phone. This is where the HTML, CSS, and JavaScript actually run. It draws the page, and it's what `candidat-profil.js`, `offres.js`, and every other file in `assets/js/pages/` execute inside.
- The server — a separate machine (in development, your own PC running `php -S`; in production, a real web server) that holds the database and runs the PHP files in `api/`. The browser never touches the database directly — it can't, and by design shouldn't be able to.

Every interaction on NeoJob follows the same basic shape: the browser sends a request (“give me all published jobs”, “log me in with this password”, “save this new job posting”) to the server, the server does some work (usually involving the database) and sends back a response, and the browser updates what's on screen based on that response. This back-and-forth is called the request/response cycle, and it happens constantly — every job card you see, every login, every drag of a Kanban card is one more round trip.

WHY THIS MATTERS A request always crosses the network, even when the “network” is just localhost during development. That round trip takes real time — a few milliseconds locally, longer on a real internet connection — and that delay is the entire reason “async” exists in this codebase. We'll come back to this in section 1.7.

1.2 What HTML, CSS, and JavaScript Each Do

A quick refresher, since NeoJob leans on the distinction constantly:

- HTML is the structure — the actual elements on the page (buttons, forms, tables, divs) and their id attributes, which is how JavaScript finds them later (`document.getElementById('login-form')`, for instance).
- CSS is the appearance — colors, spacing, the dark theme, the pill-shaped badges. NeoJob keeps all of this in `assets/css/theme.css` and doesn't touch it from JavaScript in any of the pages covered in this guide.
- JavaScript is the behavior — everything that happens after the page has loaded: fetching data, reacting to clicks, validating a form, redrawing part of the page. This is where essentially all of the “logic” you'll be asked about in your defense lives.

1.3 What PHP Does

PHP is a programming language that runs only on the server, never in the browser. When your browser requests a page like `api/offres.php`, the server executes that PHP file — running queries against the database, checking who's logged in, deciding what's allowed — and sends back plain text (in NeoJob's case, always JSON — see 1.6). The browser never sees the PHP source code itself, only whatever that PHP file decided to output. This separation is exactly why the database password, the query logic, and the security checks are all safe from anyone opening the browser's developer tools: none of it ever gets sent to the client.

1.4 Databases, SQL, and PDO — the Short Version

NeoJob's data (users, job offers, applications, messages...) lives in a MySQL/MariaDB database — permanent storage, organized into tables (`utilisateurs`, `offres`, `candidatures`, and so on), each table a grid of rows and columns. SQL (Structured Query Language) is how you ask a database for data or tell it to change something — e.g. `SELECT * FROM offres WHERE statut = 'publiee'` means “give me every row from the `offres` table where the `statut` column equals 'publiee'”.

PDO (PHP Data Objects) is PHP's built-in toolkit for talking to a database safely. Every single `api/*.php` file in NeoJob starts with `require 'config.php'`, which creates one shared PDO connection object called `$pdo` (see Part 3.1). Every query in the project goes through prepared statements — `$pdo->prepare('SELECT * FROM utilisateurs WHERE email = ?')` followed by `->execute([$email])` — rather than gluing the email directly into the SQL text. This isn't a style preference: building SQL by concatenating raw strings is how SQL injection attacks happen; prepared statements make that class of attack essentially impossible, because the database always treats the `?` placeholder as pure data, never as part of the command.

1.5 What Is an API — and Why Does NeoJob Have One?

API stands for Application Programming Interface — a fixed, agreed-upon set of “doors” a program exposes so other programs can ask it to do things, without needing to know how it does them internally. NeoJob's API is the `api/` folder: seventeen PHP files, each exposing a small set of actions (get all jobs, create an application, approve a review...) that the front-end JavaScript is allowed to call.

Here's the analogy that makes it click: think of the database as a restaurant's kitchen, and the API as the waiter. You (the browser) never walk into the kitchen and start cooking yourself — you tell the waiter what you want, the waiter relays it to the kitchen using a system only staff understand, and brings back a plate. You don't need to know how the kitchen is organized, and — critically — you're not allowed to know, because letting every customer wander into the kitchen would be chaotic and dangerous. The waiter (the API) is the only thing allowed to touch the kitchen (the database) directly.

Concretely, in NeoJob, having an API layer between the browser and the database buys three specific things:

- **Security.** The database password, the actual SQL, and the rules about who's allowed to do what (see `require_role()` in Part 3.1) live only on the server. Someone opening their browser's dev tools can see the JavaScript, but never the PHP or the database credentials behind it.
- **A stable contract.** The front-end only needs to know “if I call `GET /api/offres.php?action=getPublished`, I get back a list of published jobs shaped like `{id, titre, ville, ...}`”. What database engine sits behind that, what the actual table structure looks like, how the query is optimized — none of that is the front-end's concern, and it can all change later without breaking any page.
- **It's literally how this project evolved.** Read the project memory: NeoJob was originally built front-end-first against a fake, fully in-browser database (an object called `NeoDB`, backed by `localStorage`). When the real PHP/MySQL backend was built, the front-end code didn't have to be rewritten from scratch — `assets/js/db.js` kept every function name identical (`Jobs.getAll()`, `Applications.updateStatus()`, ...) and only the inside of each function changed, from a `localStorage` read to a real network call through the API. That migration would have been far harder without a clean API boundary already implied by the code's shape.

ONE MORE TERM: “REST” REST is the specific style of API NeoJob uses: each PHP file represents one type of “resource” (`offres.php` = job offers, `candidats.php` = candidates...), and the HTTP method used (GET/POST/PUT/DELETE) signals the kind of operation — GET to read, POST to create, PUT to update, DELETE to remove. NeoJob layers a `?action=...` query parameter on top for operations that don't fit neatly into plain CRUD (like `?action=approve` or `?action=recordView`) — you'll see this pattern in literally every file in Part 3.

1.6 JSON — the Language Between Front-End and Back-End

JSON (JavaScript Object Notation) is a plain-text way of writing structured data — objects with named fields, arrays, strings, numbers, booleans — that both JavaScript and PHP can read and write natively. It's the format every single response from NeoJob's API is sent in. A call to GET `/api/offres.php?id=1` doesn't send back an HTML page — it sends back something like:

A real API response, shortened for readability

```
{
  "id": 1,
  "titre": "Développeur Front-End React",
  "ville": "Casablanca",
  "salaireMin": 8000,
  "salaireMax": 13000,
  "competences": ["JavaScript", "React", "CSS"]
}
```

On the PHP side, this text is produced by `json_encode($data)` (see `respond()` in `api/helpers.php`, Part 3.1). On the JavaScript side, it's turned back into a real object via `res.json()` (see `apiFetch()` in `db.js`, Part 2.1) — after that call, `job.titre`, `job.ville`, etc. are ordinary JavaScript properties, exactly as if you'd typed the object yourself.

1.7 Synchronous vs. Asynchronous — Why “await” Exists

This is the concept that trips up almost everyone the first time, so let's go slowly.

Synchronous code runs one line at a time, in order, and each line finishes completely before the next one starts. Almost all the JavaScript you've probably written before — adding two numbers, changing an element's text — is synchronous, and it's effectively instant, so this never causes a problem.

But a network request (calling the API) is not instant. Between the moment the browser sends “give me all published jobs” and the moment the server's answer arrives, real time passes — the request has to travel to the server, the server has to query the database, and the answer has to travel back. If JavaScript simply froze and waited during that time, the entire page — every button, every scroll — would lock up until the network replied. Browsers refuse to work that way.

So instead, a function that starts a network request returns immediately with a placeholder object called a Promise — literally a stand-in that says “I don't have your data yet, but I promise to eventually either succeed with a result or fail with an error.” The rest of the page keeps working normally while that promise is pending in the background. The question then becomes: how does your code say “now wait for that answer before doing the next step, without freezing everything else”? That's exactly what `async` and `await` are for.

- Marking a function `async` lets you use the word `await` inside it. It also means calling that function always gives you back a Promise, even if you don't explicitly build one — JavaScript wraps whatever you return.
- `await`, used in front of a Promise, pauses only that one function at that exact line, until the Promise finishes — without freezing the browser or any other code running elsewhere on the page. Once the Promise resolves, execution continues on the next line with the actual result in hand, not the Promise wrapper.

THE #1 BEGINNER BUG The single most common bug this causes for beginners: forgetting the word `await` in front of a call to an `async` function. You don't get an error — you just get the Promise object itself instead of the data, and if you then try to read a property off it (like `someJobs.length`), you'll either get `undefined` or a crash, because you're reading a property off a Promise wrapper, not off the actual array inside it. Part 4 points out a real, live example of exactly this bug in NeoJob's own code (`a-propos.html`) — a genuinely useful example to bring up if you're asked “what would you improve?” in your defense.

1.8 A Careful, Real-Code Walkthrough of `async/await`

Let's trace the simplest real function in the whole project, `apiGet()`, and then the function that calls it, to see the mechanics with actual NeoJob code rather than an abstract example.

assets/js/db.js

```
async function apiFetch(url, options){
  const res = await fetch(url, options);
  let data = null;
  try{ data = await res.json(); }catch(e){ /* empty body */ }
  if (!res.ok){
    const err = new Error((data && data.error) || `Erreur réseau (${res.status})`);
    err.status = res.status;
    throw err;
  }
  return data;
}

function apiGet(url){ return apiFetch(url); }
```

Reading this line by line:

1. `fetch(url, options)` starts the actual network request and immediately returns a Promise. `await` pauses `apiFetch()` right there — not the whole browser, just this one function — until the server responds. When it does, `res` holds the raw HTTP response (status code, headers, and a body that hasn't been read yet).
2. `res.json()` reads and parses the response body as JSON — this is also asynchronous (reading a network body takes a moment too), so it's also awaited. The `try/catch` around it exists because a few responses

(like some 204 “no content” replies) have no body at all, and calling `.json()` on an empty body would otherwise throw.

3. If the HTTP status wasn't a success code (`res.ok` is false — e.g. a 401 for a bad login, or a 403 for the wrong role), the function builds a proper `Error` object and throws it, rather than returning it — this lets calling code use an ordinary `try/catch` to react to failures.
4. Otherwise, it returns the parsed data.

Now look at `apiGet(url)`: it has no `async` keyword and no `await` inside it at all — and yet calling it still gives you a `Promise`, because it simply returns whatever `apiFetch(url)` returns, and `apiFetch` is `async`. This is worth sitting with: a function doesn't need the `async` keyword itself to “count” as asynchronous — it just needs to return a `Promise`, however that `Promise` was produced. You'll see this exact pattern (a plain function that just forwards to an `async` one) throughout `db.js`, in every single method of `Jobs`, `Companies`, `Candidates`, and so on.

Finally, here's what it looks like from the calling side, in a real page (`candidat/dashboard.html`'s script):

assets/js/pages/candidat-dashboard.js (excerpt)

```
const session = await Auth.guardPage('candidat', '..../');
if (!session) return;

const candidate = await Candidates.getByUserId(session.userId);
const [applications, favorites] = await Promise.all([
  Applications.getByCandidate(candidate.id),
  Favorites.getByCandidate(candidate.id)
]);
```

Every one of those `awaits` pauses this script at that line until the corresponding network call finishes, and every one of them gives back real, usable data on the next line — `session` is an actual object with `.userId` on it, not a `Promise`; `candidate` is an actual candidate record; `applications` and `favorites` are actual arrays you can call `.length` or `.filter()` on. The `Promise.all([...])` on the last two lines is a specific, very common pattern worth calling out on its own: it fires off both requests at the same moment (instead of waiting for the first to fully finish before even starting the second) and waits for both together — since neither request depends on the other's result, there's no reason to make the user wait twice as long by doing them one after another. You'll see this exact `Promise.all([...])` pattern dozens of times across the codebase from this point on.

WHY EVERY PAGE STARTS WITH `(async () => { ... })()`: `await` only works inside a function marked `async`. Since a page's top-level script can't just be `async` on its own, essentially every page in `NeoJob` wraps its entire script in an “immediately invoked `async` function”: `(async () => { ...all the code... })()`; — you'll see this exact wrapper at the top of literally every page script from Part 4 onward. It's just a small, self-running container

that exists purely so the word `await` is legal to use throughout the file.

One more real pattern worth knowing before Part 2, because it looks like `async/await` but isn't built from a network call at all — NeoJob's own “are you sure?” confirmation dialog:

assets/js/main.js — `NeoUI.confirmDialog()` (condensed)

```
confirmDialog(message, title){
  return new Promise((resolve) => {
    ... // show a Bootstrap modal with OK / Cancel buttons
    onOk = () => { modal.hide(); resolve(true); };
    onHide = () => { resolve(false); };
    ...
  });
}
```

This function has no `async` keyword and touches the network exactly zero times — instead it manually builds a new `Promise((resolve) => {...})` and calls `resolve(true)` or `resolve(false)` from inside a button's click handler, whenever the human actually clicks something. From the calling page's point of view, though, it behaves identically to awaiting a network call: `const ok = await NeoUI.confirmDialog('Delete this?'); if (ok) { ... }`. This is the clearest proof that `await` isn't only “for fetch” — it works with any `Promise`, and a `Promise` can represent waiting on absolutely anything eventual, including a human clicking a button. You'll meet this exact function again in almost every admin and recruteur page in Part 4, right before every delete action.

1.9 Sessions and Cookies — How the Site Remembers You're Logged In

HTTP, the protocol browsers and servers speak, has no built-in memory — by default, the server treats every single request as if it's from a total stranger, even the very next request from the same browser one second later. A session is how the server keeps track of “this browser is logged in as user #4” across many separate requests.

In NeoJob, this works through PHP's native session mechanism: when `session_start()` runs (the very first line of `api/session.php`, included by every endpoint), PHP either starts a new session or resumes an existing one, based on a small session-id cookie the browser automatically sends with every request to the same site. On a successful login, `api/auth.php` writes `$_SESSION['user'] = ['userId' => ..., 'email' => ..., 'role' => ...]` — and from that point on, any endpoint can call `current_user()` to read that same `$_SESSION['user']` back, on any later request, without the browser needing to resend the password. `require_role()` (also in `session.php`) is built directly on top of this: it calls `current_user()`, and rejects the request with a 401 or 403 if there's no session or the wrong role. You'll see `require_role(...)` at the top of nearly every protected action in Part 3.

A HISTORICAL NOTE WORTH KNOWING Earlier in the project's life, “being logged in” was faked entirely inside the browser via `localStorage` — there was no real server-side session at all. That's no longer true: as the comment at the top of `assets/js/auth.js` puts it, “the real session now lives server-side in a PHP session (cookie-based), not in `localStorage`.” The one thing that still legitimately uses `localStorage` today is unrelated to login — it's `NeoTheme`, which just remembers your dark/light mode preference (see Part 2.5). Don't confuse the two if you're asked about it.

1.10 Putting It All Together — One Full Request, Start to Finish

Before diving into the code itself, here is the entire chain of events for one single, concrete action — a candidate clicking “Postuler” (Apply) on a job — with every concept above appearing in its actual place:

5. The candidate clicks the “Postuler” button on `offre-detail.html`. A JavaScript click listener defined in `assets/js/pages/offre-detail.js` fires.
6. That listener calls `await Applications.create({...})` — a function defined in `db.js`.
7. `Applications.create()` calls `apiPost('/api/candidatures.php', body)`, which builds a POST request with the application data encoded as JSON in the request body, and calls `apiFetch()` under the hood.
8. `apiFetch()` calls the browser's `fetch()`, which sends an actual HTTP request across the network (even if that network is just `localhost`) to the PHP server, and returns a Promise immediately — the rest of the page keeps working while this is in flight.
9. On the server, `php -S` routes the request to `api/candidatures.php`. That file runs `require_role('candidat')` (`session.php`) to confirm someone's actually logged in as a `candidat` — using the session cookie the browser sent automatically.
10. The file reads the JSON body via `json_body()` (`helpers.php`), then runs a prepared `INSERT INTO candidatures (...)` statement through PDO (`$pdo`, from `config.php`) — this is the one moment the database itself is actually touched.
11. The file calls `respond($newApplication, 201)` (`helpers.php`), which `json_encodes` the new record and sends it back as the HTTP response body, with status code 201.
12. Back in the browser, `apiFetch()`'s `await fetch(...)` finally resolves with that response; `await res.json()` parses it back into a real JavaScript object.
13. Control returns, one function call at a time, back up through `apiFetch()` → `apiPost()` → `Applications.create()` → the original click handler in `offre-detail.js` — each `await` along the way having paused only that link in the chain.
14. The click handler's code (which had been paused on its own `await`) now runs its next line: it shows a success toast and re-renders the page's “apply” button as “already applied”.

Every single page covered in Part 4 is built from variations of this exact same chain. Once this one makes sense, the rest of this guide is mostly just “which specific function gets called, and why”, not new mechanics.

Part 2 — The Shared Toolkit

Seven files get loaded on almost every page, in this exact order, right before the page's own script: `db.js`, `notifications.js`, `auth.js`, `components.js`, `main.js`, and — on pages that need them — `charts.js` and `quiz-data.js`. This part explains every one of them, function by function, once and thoroughly, so that every page walkthrough in Part 4 can simply say “this calls `X()`” and you'll already know exactly what that means.

2.1 `db.js` — Talking to the API

This is the single most important file in the whole front-end: every place any page reads or writes data, it goes through here. The file's own header comment is worth quoting directly, because it explains the file's entire history in two sentences: it used to read and write a fake in-browser database (an object called `NeoDB`, backed by `localStorage`), and it was rewritten to call the real PHP API instead — while deliberately keeping every function's name identical, so no other file in the project had to change just because the data source changed underneath it.

`apiFetch()`, `apiGet()`, `apiPost()`, `apiPut()`, and `apiDelete()` — the network plumbing shared by everything else in this file — were already walked through in detail in section 1.8. Everything below them is a set of plain objects, one per “resource” (`Jobs`, `Companies`, `Candidates`, ...), each just a thin, readable wrapper around those five functions. The table below lists every single one.

Table 1 — Jobs — job postings (offres)

Method	What it does
<code>getAll()</code>	GET every job, any status — used by admin pages.
<code>getById(id)</code>	GET one job by id.
<code>getPublished()</code>	GET only jobs with <code>statut = 'publiee'</code> — the public job board.
<code>getPending()</code>	GET jobs awaiting moderation (<code>statut = 'en_attente'</code>).
<code>getByCompany(companyId)</code>	GET every job posted by one company.
<code>create(job)</code>	POST a new job posting (starts out pending, see Part 3).
<code>update(id, changes)</code>	PUT a partial update — merges <code>{id}</code> into <code>changes</code> first.
<code>remove(id)</code>	DELETE a job posting.

approve(id) / reject(id)	PUT — admin moderation actions.
recordView(id)	POST — increments the job's view counter.

Table 2 — Companies — recruiter company profiles (entreprises)

Method	What it does
getAll() / getById(id)	Standard reads.
getByUserId(userId)	Finds the company profile belonging to a logged-in recruiter.
create(c) / update(id, changes)	Standard writes.
upgradeToPremium(id)	PUT — marks a company as premium plan.
activeJobCount(id)	GET — how many active jobs this company currently has.
setVerified(id, verified)	PUT — admin toggles the “verified company” badge.
hasReachedFreeLimit(id)	async — see the walkthrough just below this table.

hasReachedFreeLimit is the one method in this object that actually needs the async keyword rather than just forwarding a Promise, and it's a good second example after apiFetch:

assets/js/db.js

```
const FREE_PLAN_JOB_LIMIT = 6;

hasReachedFreeLimit: async (id) => {
  const company = await Companies.getById(id);
  if (!company || company.plan === 'premium') return false;
  const count = await Companies.activeJobCount(id);
  return count >= FREE_PLAN_JOB_LIMIT;
}
```

It needs real async/await (not just “return apiGet(...)”) because it has to branch on the result of the first call before deciding whether to even make the second call — a premium company never needs its job count checked at all, so the function returns early. You literally cannot write that “check the first answer before deciding what to ask next” logic with a bare return statement.

Table 3 — Candidates — candidate profiles (candidats)

Method	What it does
getAll() / getById(id)	Standard reads.
getByUserId(userId)	Finds the profile for a logged-in candidat.

create(c) / update(id, changes)	Standard writes.
verifySkill(id, skillName)	PUT — marks one skill as verified after a passed quiz.

Table 4 — Categories, Competences, Applications, Interviews, Favorites, Payments, Reviews, JobAlerts, Messages, Users, Uploads

Object	Notable methods
Categories	getAll/getById/create/update/remove — plain CRUD for the job-category list.
Competences	Same CRUD, plus findOrCreateByName(name) — async; looks up a skill by exact name and creates it if it doesn't exist yet, always returning the canonical name.
Applications	getAll/getById/getByCandidate/getByJob/hasApplied/create/updateStatus(id, statut)/remove — the whole applicant lifecycle.
Interviews	getAll / getByApplication / getByCandidate / getByCompany / create / update / selectSlot(id, slot) / cancel / remove.
Favorites	getByCandidate / isFavorite / toggle(candidateId, jobId) — toggle asks the server to add or remove in one call; the server decides which.
Payments	getAll / getByCompany / create — see Part 4's recruteur/paiement.html for how this simulates a checkout.
Reviews	getAll/getById/getByCompany(id,{onlyPublished})/getPending/hasReviewed/create/approve/reject/remove/getAverageRating(companyId).
JobAlerts	getByCandidate/create/remove, plus matches() and getMatchingJobs() — see below, they're worth their own explanation.
Messages	getByApplication(applicationId) / create — the per-application chat thread.
Users	getAll/getById/getByEmail/update/remove/setStatus(id,status)/markNotificationsSeen(id) — the account layer, separate from the profile data in Candidates/Companies.
Uploads	upload(file, kind) — see below, the only place in db.js that doesn't use apiFetch at all.

JobAlerts.matches(alert, job) deserves its own look, because — unlike almost everything else in this file — it is deliberately not async at all:

assets/js/db.js

```
matches: (alert, job) => {
  if (alert.q){
    const q = alert.q.toLowerCase();
```

```

const inTitle = job.titre.toLowerCase().includes(q);
const inSkills = (job.compétences || []).some(c => c.toLowerCase().includes(q));
if (!inTitle && !inSkills) return false;
}
if (alert.ville && !job.ville.toLowerCase().includes(alert.ville.toLowerCase())) return false;
if (alert.compétence && !(job.compétences || []).includes(alert.compétence)) return false;
...
return true;
}

```

It never touches the network — it just compares two already-in-memory objects, one criterion at a time, bailing out with `false` the moment any set criterion fails to match. The related `getMatchingJobs(alert)` is `async`, precisely because it does need the network first: `await Jobs.getPublished()`, and only then filters that list with the synchronous `matches()` function above. This pair is a clean, small example of the general rule: fetch with `await`, then compute in plain, ordinary, synchronous JavaScript.

And `Uploads.upload` — the one exception to “everything goes through `apiFetch`”:

assets/js/db.js

```

async upload(file, kind){
  const form = new FormData();
  form.append('file', file);
  form.append('kind', kind || 'photo');
  const res = await fetch('/api/upload.php', { method: 'POST', body: form });
  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Échec du téléversement. ');
  return data; // { path, name }
}

```

This bypasses `apiPost()` on purpose: `apiPost()` always sets `Content-Type: application/json`, which would be wrong for a file upload — a `FormData` body needs the browser to set its own multipart boundary header automatically, so this function calls `fetch()` directly instead, keeping the same “await it, throw on failure” shape as everything else by hand.

2.2 `auth.js` — Login, Sessions, and Guarding Pages

This file's header comment says it plainly: “the real session now lives server-side in a PHP session (cookie-based), not in `localStorage`.” Every method on the `Auth` object is `async`.

Table 5 — The `Auth` object

Method	What it does
<code>login(email, password)</code>	POSTs to <code>/api/auth.php?action=login</code> . Returns <code>{ok:false, error}</code> or <code>{ok:true, user}</code> — never throws, so callers just check <code>.ok</code> .

register(payload)	Same result shape as login, POSTs the whole signup form.
logout()	GET /api/auth.php?action=logout — no return value, purely a side effect (ends the session).
changePassword(current, next)	Same {ok, error} shape as login.
current()	GET /api/auth.php?action=current. Returns the session object, or null if nobody's logged in. The single most-called method in the whole codebase.
isLoggedIn()	return !(await this.current()) — awaits first, then converts to a real boolean (see the note below on why the await here is not optional).
guardPage(roles, rootPrefix)	The page-protection function — see below.
spaceHome(role)	NOT async — pure string logic, returns 'candidat/dashboard.html' / 'recruteur/dashboard.html' / 'admin/dashboard.html' / 'index.html'.

A REAL BUG THIS WOULD CAUSE IF THE AWAIT WERE MISSING isLoggedIn() is a good one-line proof of why await matters: !(Auth.current()) — without the await — would always evaluate to true, because a pending Promise object is itself truthy in JavaScript, regardless of whether the login check inside it eventually resolves to a real session or to null. Only awaiting first, so !! is applied to the resolved value and not the Promise wrapper around it, makes the boolean actually meaningful.

guardPage is the function almost every protected page in Part 4 starts with:

assets/js/auth.js (as used across the project)

```

async guardPage(roles, rootPrefix = ''){
  const session = await this.current();
  if (!session) { window.location.href = rootPrefix + 'login.html'; return null; }
  if (roles){
    const allowed = Array.isArray(roles) ? roles : [roles];
    if (!allowed.includes(session.role)) {
      window.location.href = rootPrefix + this.spaceHome(session.role);
      return null;
    }
  }
  return session;
}

```

Called at the very top of a protected page as const session = await Auth.guardPage('candidat', '..../'); if (!session) return; — the redirect (if any) has already started by the time that if runs; the return simply

stops the rest of the page's script from doing pointless work (fetching data, building HTML) for a page the browser is about to navigate away from anyway. Passing null instead of a role string (as `parametres.html` does) means “any logged-in user, any role, is allowed” — the only page in the whole project that does this, since account settings are shared across all three roles.

2.3 components.js — Header, Sidebar, and Reusable Cards

This file's own comment explains its design rule up front: `renderHeader/renderSidebar` are async (they need to read the session and notifications over the API first), but `jobCard/candidateCard` are kept synchronous on purpose, because they're called inside tight render loops (once per job, once per candidate) — awaiting inside a loop, one item at a time, would be slow and clumsy. Instead, the calling page fetches company data once up front (`Companies.getAll()`) and passes the already-resolved company into each card via `opts.company`.

Table 6 — NeoComponents — public functions

Function	async?	What it does
<code>renderHeader(opts)</code>	yes	Builds and injects the navbar into <code>#site-header</code> : logo, nav links, and — if logged in — the notification bell and user menu, or — if not — login/register buttons. Wires the logout button and search-trigger click handlers.
<code>renderFooter(opts)</code>	no	Injects the static footer into <code>#site-footer</code> . No session, no API calls — pure static HTML from a template, a useful contrast case for “not everything needs to be async.”
<code>renderSidebar(opts)</code>	no	Injects the role-specific nav menu into <code>#site-sidebar</code> (desktop) and <code>#site-sidebar-mobile</code> — a static, role-keyed lookup table, no fetch at all. The caller must already know the role (it just ran <code>guardPage</code> , after all) and passes it in.
<code>jobCard(job, opts)</code>	no	Builds one job-listing card's HTML. If <code>opts.company</code> isn't given, falls back to a generic “Enterprise” placeholder rather than fetching one itself.
<code>candidateCard(candidate, opts)</code>	no	Builds one candidate-directory card's HTML, styling verified skills differently from unverified ones.
<code>companyBadgeHtml(logoUrl, name, style)</code>	no	A logo image if one exists, otherwise a colored initials badge.
<code>isFeaturedJob(company)</code>	no	<code>!!(company && company.plan === 'premium')</code> — whether a job's

		company should be visually pinned/highlighted as “sponsored”.
--	--	---

2.4 notifications.js — The Notification Bell

This file has no database table of its own — it's a pure aggregation layer that stitches together data from many db.js objects to answer “what's new since you last checked?”, separately for each role.

Notifications.getForSession(session, rootPrefix) is the single entry point (called from renderHeader() above); it dispatches to one of three private methods depending on role:

- `_forCandidat` — looks for application-status changes, newly proposed interviews, job-alert matches, and new recruiter messages, all newer than the candidate's `lastSeenNotifAt` timestamp.
- `_forRecruteur` — looks for new applications, newly confirmed interviews, and new candidate messages, across every job that recruiter's company has posted.
- `_forAdmin` — always shows the live moderation queue (pending jobs + pending reviews); unlike the other two, this one has `dismissible: false`, meaning opening the bell never “clears” it — it always reflects however many items are genuinely still pending.

The `candidat` version is the most instructive one for understanding real async composition, because it deliberately mixes two different strategies in the same function:

assets/js/notifications.js — `_forCandidat()` (condensed)

```
// step 1 – kicked off, but NOT yet awaited: an array of PENDING promises
const statusItems = applications
  .filter(a => a.statut !== 'en_attente' && isNewerThan(a, lastSeen))
  .map(async a => {
    const job = await Jobs.getById(a.jobId);
    return { message: `...`, date: a.updatedAt, url: `...` };
  });

// step 2 – same idea, another pending array
const interviewItems = interviews
  .filter(iv => iv.statut === 'proposee' && isNewerThan(iv, lastSeen))
  .map(async iv => { const job = await Jobs.getById(iv.jobId); return { ... }; });

// step 3 – THIS is where they actually get resolved, merged, and sorted
const items = (await Promise.all(statusItems.concat(interviewItems)))
  .concat(alertItems, messageItems)
  .sort((a, b) => new Date(b.date) - new Date(a.date));
```

`array.map(async item => {...})` always produces an array of Promises, never an array of resolved results — even though it reads like an ordinary `.map()`. Nothing forces you to `await` immediately, and this function deliberately doesn't: it builds up several separate pending-promise arrays first (steps 1 and 2), and only calls `Promise.all(...)` once, at the very end (step 3), letting every one of those `Jobs.getById()` calls actually fire off and resolve concurrently, in parallel, rather than one at a time. `Notifications.markSeen(session)` is the other half of this file — it updates the server-side

lastSeenNotifAt timestamp (via Users.markNotificationsSeen) whenever the bell dropdown is opened, but only for candidat/recruteur (never admin, per the dismissible: false rule above).

2.5 main.js — NeoUI, NeoTheme, and the Command Palette

The largest of the shared files, and mostly a grab-bag of small, focused helpers used everywhere. Grouped by purpose:

Feedback to the user

Function	What it does
NeoUI.toast(message, type)	Shows a temporary Bootstrap toast (success/danger/info/warning) in the corner, auto-dismissing after 3.2 seconds. Not async — fires and returns immediately; the timing is handled by Bootstrap's own internal clock.
NeoUI.confirmDialog(message, title)	Manually wraps a Bootstrap modal in new Promise((resolve) => {...}) — see the full walkthrough in section 1.8. Every destructive action in the project (delete a job, remove a user, reject a review) awaits this before doing anything irreversible.
NeoUI.confetti()	A short, on-brand confetti burst built from 60 plain <div> elements and CSS animation — no external library. Fires on a passed quiz, a successful hire, and a successful account registration.

Formatting

Function	What it does
formatMoney(n)	1300 → "1 300 MAD" (French locale grouping + currency label).
formatDate(d) / formatDateTime(d)	Human-readable French date, with or without time.
statusPill(map, key)	Renders a colored pill from a lookup map — used with the two constants below.
JOB_STATUS / APP_STATUS	Lookup objects mapping raw status strings (e.g. 'en_attente') to a French label + a CSS pill class.
APP_PIPELINE_ORDER	['en_attente','preselection','entretien','offre','embauchee','refusee'] — the exact order that drives the Kanban board's column order in Part 4.3.

<code>escapeHtml(str)</code>	Safely escapes user-provided text before it's inserted into HTML — used constantly (candidate names, job titles, messages) to prevent XSS.
------------------------------	--

Avatars, ratings, and files

Function	What it does
<code>avatarHtml(photoUrl, name, style)</code>	A circular photo if one exists, otherwise a colored initials badge (via <code>initials(name)</code>).
<code>starsHtml(value)</code>	A read-only 5-star display, supporting half-stars.
<code>starInputHtml(id, value) + wireStarInput(id)</code>	A clickable 5-star input — the HTML renderer and the click/hover wiring are deliberately two separate functions, since the elements don't exist in the live page yet at the moment the HTML string is generated.
<code>readFileAsDataURL(file)</code>	Another hand-built Promise (like <code>confirmDialog</code>) — wraps the older, callback-based <code>FileReader</code> browser API so it can be awaited.
<code>matchScore(candidateSkills, jobSkills)</code>	% overlap between what a candidate has and what a job wants — returns null (not 0) if the job lists no requirements at all.

The command palette (Ctrl+K)

NeoPalette is a self-contained search-everything overlay, available on every page via Ctrl+K (or Cmd+K). Typing into it calls an `async renderResults(query)` that — depending on what's typed and who's logged in — searches `Jobs.getPublished()`, `Companies.getAll()`, and (only for `recruteur/admin`, matching the same access rule as the candidate-search page in Part 4.3) `Candidates.getAll()`, building up to three labeled result sections.

2.6 charts.js — Dashboard Charts

A thin theming wrapper around the third-party Chart.js library, loaded only on pages that actually show a chart (the recruiter and admin dashboards). `NeoCharts.applyTheme()` sets Chart.js's global color/font defaults once to match NeoJob's dark palette. `NeoCharts.barChart(canvasId, labels, data, opts)` builds one themed bar chart — but deliberately inside a `setTimeout(...,30)`, not synchronously:

assets/js/charts.js (comment, verbatim)

```
// Builds the chart on a deferred tick, not synchronously. A bar chart created the
```

```
// instant the page script runs can measure a container that hasn't finished its
// flex layout yet, and never recovers. setTimeout sidesteps that reliably.
```

“DELAYED” IS NOT THE SAME AS “ASYNC” This is worth calling out precisely because it looks like it should be “async” but isn't: `barChart()` has no `async` keyword and returns nothing usable — there's no way to await it and know the chart is actually finished drawing. `setTimeout` schedules something for later; it does not, by itself, make a function part of the Promise/async system. Delayed and asynchronous (in the Promise sense) are two different things in JavaScript, and this function is a clean real-world example of the first without the second.

2.7 quiz-data.js — the Skill Verification Question Bank

The simplest file in the project: no functions at all, just one constant object, `NeoQuizData`, mapping a skill name (exactly as stored in the competences table) to an array of three multiple-choice questions, each shaped `{q, options, answer}` where `answer` is the index of the correct option. It's read by `candidat/profil.html`'s quiz feature (see Part 4.2) and by nothing else in the project.

A REAL DATA BUG TO KNOW ABOUT Worth knowing in case you're asked: the 'Java' and 'Kotlin' entries currently contain the exact same three questions as 'SEO' — word for word (what SEO stands for, the `<title>` tag, and backlinks), not anything about Java or Kotlin. This is a genuine copy-paste content bug in the seed data, not a matter of opinion — a good, honest answer if you're asked “is everything in the project perfect?” during your defense.

Part 3 — The Backend: Inside the API

This part walks through every one of the seventeen files in `api/`. Three of them (`config.php`, `session.php`, `helpers.php`) are pure shared infrastructure that every other file leans on; the other fourteen each manage exactly one resource — `offres.php` manages job postings, `entreprises.php` manages companies, and so on — and are covered as tables of every action they expose, in the same format each time: the action, the HTTP method, what it actually does, and whether it's protected by a login/role check.

3.1 The Three Shared Files

Every single one of the other fourteen files begins with the identical three lines: `require 'config.php';` `require 'helpers.php';` `require 'session.php';` — this is how every endpoint gets its database connection, its response helpers, and its auth check, without repeating any of that logic.

api/config.php

```
$pdo = new PDO(
    "mysql:host=$DB_HOST;port=$DB_PORT;dbname=$DB_NAME;charset=utf8mb4",
    $DB_USER, $DB_PASS,
    [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
      PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC ]
);
```

`PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` means a failed query throws a real PHP exception instead of silently returning false — this is exactly why none of the other sixteen files ever check `->execute()`'s return value; a failure would already have crashed loudly. `PDO::FETCH_ASSOC` means every `->fetch()/->fetchAll()` call returns rows as `$row['column_name']` arrays everywhere in the project, with no per-call configuration needed.

api/session.php

```
session_start();

function current_user(){
    return $_SESSION['user'] ?? null;
}

function require_role($roles = null){
    $user = current_user();
    if (!$user) respond_error('Non authentifié.', 401);
    if ($roles !== null) {
        $allowed = is_array($roles) ? $roles : [$roles];
        if (!in_array($user['role'], $allowed, true)) respond_error('Accès refusé.', 403);
    }
    return $user;
}
```

This is the security checkpoint used across the whole API — covered in depth in section 1.9. Note that `respond_error()` (`helpers.php`, below) calls `exit` — so a failed check here genuinely stops the rest of that file's code from running at all, not just from returning correct data.

api/helpers.php

```
function json_body(){
    $data = json_decode(file_get_contents('php://input'), true);
    return is_array($data) ? $data : [];
}

function respond($data, $code = 200){
    http_response_code($code);
    echo json_encode($data);
    exit;
}

function respond_error($message, $code = 400){
    http_response_code($code);
    echo json_encode(['error' => $message]);
    exit;
}
```

`json_body()` is how every POST/PUT handler reads the JSON the front-end sent — `$in = json_body();` `$in['titre']` and so on. `respond()` and `respond_error()` are the only two ways any endpoint ever finishes: encode the result as JSON, print it, and exit — this is why every action in every file below ends with one or the other.

3.2 auth.php — Login and Account Creation

Manages the `utilisateurs` table and, on registration, the linked `candidats` or `entreprises` row in the same breath.

Table 7 — api/auth.php — actions

Action	Method	What it does	Auth?
current	GET	Returns <code>current_user()</code> as-is — this is how the front-end's <code>Auth.current()</code> knows who's logged in.	None (that's the point)
login	POST	Finds the user by email, checks <code>password_verify()</code> against the stored bcrypt hash, rejects blocked accounts, then writes <code>\$_SESSION['user']</code> .	None (entry point)
logout	any	<code>\$_SESSION = []</code> ; <code>session_destroy()</code> ; — fully ends the session.	None
register	POST	Validates role is 'candidat' or 'recruteur' (never 'admin' via self-signup), checks for a duplicate email, then INSERTs into <code>utilisateurs</code> AND the matching profile table inside one	None (entry point)

		database transaction — see below.	
changePassword	POST	Verifies the current password, then updates the hash.	Any logged-in user

The register action is the clearest example in the whole codebase of an atomic multi-table write — a user account is meaningless without its role-specific profile row, so the two INSERTs either both succeed or neither does:

api/auth.php

```
$pdo->beginTransaction();
try {
    $stmt = $pdo->prepare('INSERT INTO utilisateurs (email, password_hash, role) VALUES (?, ?, ?)');
    $stmt->execute([$email, password_hash($password, PASSWORD_DEFAULT), $role]);
    $userId = (int)$pdo->lastInsertId();

    if ($role === 'candidat') {
        $pdo->prepare('INSERT INTO candidats (utilisateur_id, prenom, nom) VALUES (?, ?, ?)');
    } else {
        $pdo->prepare('INSERT INTO entreprises (utilisateur_id, nom, secteur) VALUES (?, ?, ?)');
    }
    $stmt->execute([$userId, $in['prenom'] ?? '', $in['nom'] ?? '']);
    $pdo->commit();
} catch (Exception $e) {
    $pdo->rollBack();
    respond_error("Erreur lors de la création du compte.", 500);
}
```

`password_hash($password, PASSWORD_DEFAULT)` is PHP's built-in, officially-recommended way to hash a password — it uses `bcrypt` and automatically generates and stores a random salt as part of the resulting hash string, so two users with the same password never end up with the same stored hash.

3.3 The Join-Table Pattern — One Idea That Repeats Constantly

Before the remaining files, one recurring pattern is worth explaining once, clearly: several pieces of data in NeoJob are “many-to-many” — a job can require many skills, and each skill can be required by many jobs. That relationship can't live as a simple column on either table, so it lives in its own small join table (`offre_compétences`, `candidat_compétences`, `entreprise_photos`, `offre_compétences`, and so on) — just pairs of ids (and sometimes one extra flag, like `verifiee`) linking two other tables together.

api/offres.php

```
function offre_compétences($pdo, $offreId){
    $stmt = $pdo->prepare('SELECT c.nom FROM offre_compétences oc JOIN compétences c ON c.id = oc.compétence_id WHERE oc.offre_id = ?');
    $stmt->execute([$offreId]);
    return $stmt->fetchAll(PDO::FETCH_COLUMN);
}
```

Reading a job's required skills means joining `offre_competences` (which only stores id pairs) against `competences` (which stores the actual skill name), then pulling out just the name column. Writing a job's skills means replacing the whole set at once — delete every existing link row for that job, then re-insert one row per submitted skill name, creating the skill in the master competences table first if it doesn't already exist (a “find-or-create” pattern you'll see repeated, slightly differently, in three separate files). Once you recognize this shape once, you'll see it again in `entreprises.php` (`entreprise_photos`), `candidats.php` (`candidat_competences` — with the extra `verifiee` flag), and `entretiens.php` (`creneaux_entretien`).

3.4 offres.php — Job Postings

Table 8 — `api/offres.php` — actions

Action	Method	What it does	Auth?
(by id)	GET	One job, fully shaped (see <code>shape_offre</code> below).	None
<code>getPublished</code>	GET	Only <code>statut='publiee'</code> jobs — the public job board.	None
<code>getPending</code>	GET	Jobs awaiting moderation.	None — see warning below
<code>getByCompany</code>	GET	All of one company's jobs.	None
(bare)	GET	Every job, any status.	None
<code>recordView</code>	POST	<code>vues = vues + 1</code> .	None
(create)	POST	Inserts a new job; <code>statut</code> is force-set to <code>'en_attente'</code> server-side regardless of what the client sends.	<code>recruteur</code>
approve / reject	PUT	Sets <code>statut</code> to <code>'publiee'</code> / <code>'rejetee'</code> .	None — see warning
(update)	PUT	Partial update from a whitelist of editable fields; replaces skills if provided.	None — see warning
(delete)	DELETE	Removes the job.	None — see warning

SECURITY GAP WORTH KNOWING This file has a real, worth-knowing security gap: only creating a job requires being logged in as a `recruteur`. Approving, rejecting, editing, and deleting a job — actions that should clearly be admin- or owner-only — have no `require_role()` call at all in the current code. If you're

asked “what would you improve for production?”, this is a genuinely good, specific answer.

`shape_offre($row, $pdo)` — the pattern repeated in every file below — converts one raw database row (snake_case columns, exactly as MySQL stores them) into the camelCase object shape the front-end expects, resolving the skills join table along the way: `{id, companyId, titre, categoryId, typeContrat, ville, remote, salaireMin, salaireMax, description, competences, statut, datePublication, vues}`.

3.5 entreprises.php — Companies

Table 9 — *api/entreprises.php* — actions

Action	Method	What it does	Auth?
(by id)	GET	One company.	None
getByUserId	GET	The company profile linked to a given account — “my company” after login.	None
activeJobCount	GET	COUNT of that company's non-rejected jobs.	None
(bare)	GET	Every company.	None
(create)	POST	Creates a company profile; plan is force-set to 'free' server-side.	recruteur
upgradeToPremium	PUT	Sets plan = 'premium'.	None — see warning
setVerified	PUT	Toggles the verified badge.	None — see warning
(update)	PUT	Partial update; if a photos array is submitted, fully replaces the <code>entreprise_photos</code> join rows to match (a full-replace, not an incremental add/remove).	None — see warning

SECURITY GAP Same pattern as `offres.php`: only creation is protected. Upgrading to premium and toggling the verified badge — both meaningful, sensitive actions — currently have no auth check.

3.6 candidats.php — Candidate Profiles

Table 10 — *api/candidats.php* — actions

Action	Method	What it does	Auth?
(by id) / getByUserId / (bare)	GET	Standard reads — the bare listing returns every candidate's full profile, including phone and CV path.	None
(create)	POST	Creates a candidate profile.	None
verifySkill	PUT	Adds (if missing) and marks one skill verified=1 for a candidate.	None
(update)	PUT	Partial update; if skills[] is submitted, fully replaces the skill list — but preserves the verified flag on any skill that was already verified and is still present in the new list.	None

THE BIGGEST SECURITY GAP IN THE API This entire file has zero `require_role()` calls anywhere — every single action, including listing every candidate's full profile, is callable by anyone, logged in or not. This is the single largest security gap in the project and a very good, concrete thing to mention if asked what you'd fix next.

The skill-preserving update logic is a nice small detail worth knowing: before replacing a candidate's skill list, the code first reads which of the current skills were already verified, then re-inserts the new list with `verifiee=1` only for names that were on that “keep verified” list — so editing your profile and re-typing a skill you'd already earned a badge for doesn't silently strip the badge.

3.7 candidatures.php — Job Applications

Table 11 — *api/candidatures.php* — actions

Action	Method	What it does	Auth?
(by id) / getByCandidate / getByJob / hasApplied / (bare)	GET	Reads — the bare listing returns literally every application in the system, cover letters included.	None
(create)	POST	Submits an application; statut is force-set to 'en_attente'.	candidat

updateStatus	PUT	Moves an application to a new pipeline stage — this is exactly what the Kanban board (Part 4.3) calls on every drag-and-drop.	None
(delete)	DELETE	Removes an application.	None

Worth knowing: deleting an application doesn't need any manual cleanup code for its related interview or messages — the database schema's own ON DELETE CASCADE foreign-key rule handles that automatically the moment the DELETE runs. This is a good, concrete example to cite if asked “how does the database enforce integrity” — it's not PHP code doing the cleanup, it's a schema-level guarantee.

3.8 entretiens.php — Interview Scheduling

The most composite file in the API: an interview's `shape_entretien()` has to reach through two other tables to fill in fields the `entretiens` table doesn't store directly — it joins `candidatures` then `offres` just to recover `jobId/candidateId/companyId`, and separately resolves every proposed time slot from the `creneaux_entretien` join table.

Table 12 — api/entretiens.php — actions

Action	Method	What it does	Auth?
(by id) / getByApplication / getByCandidate / getByCompany / (bare)	GET	Reads, several joining through <code>candidatures</code> and/or <code>offres</code> .	None
(create)	POST	Creates an interview with <code>statut='proposee'</code> and inserts every proposed slot into <code>creneaux_entretien</code> .	recruteur
selectSlot	PUT	Candidate's chosen slot — sets <code>creneau_selectionne_id</code> and auto-confirms (<code>statut='confirmer'</code>).	None
cancel	PUT	Sets <code>statut='annulee'</code> .	None
(update / reschedule)	PUT	Partial update; if <code>slots[]</code> is resubmitted, fully replaces the proposed slots and clears any prior selection.	None

3.9 favoris.php, paiements.php, alertes.php, messages.php — the Smaller Files

Table 13 — Four smaller resource files, side by side

File	Manages	Notable action	Auth?
favoris.php	Saved/bookmarked jobs (candidat ↔ offre)	toggle — one call adds or removes, server decides which.	toggle: candidat
paiements.php	Simulated payments	create — always sets statut='reussi' unconditionally; no real payment gateway is involved, by design.	create: recruteur
alertes.php	Saved job-search alerts	create — looks up (but does NOT create) a matching skill by name; if it doesn't already exist, the skill filter is silently dropped.	create/delete: candidat
messages.php	Per-application chat thread	create — the sender's role is trusted from the request body, not re-derived from the session.	create: candidat or recruteur

3.10 avis.php — Company Reviews (the best-secured file)

Manages candidate-submitted reviews of companies — multi-dimensional ratings (overall, ambiance, pay, work/life balance), free-text pros/cons, and the same pending → publiee/rejetee moderation flow used for job postings.

Table 14 — api/avis.php — actions

Action	Method	What it does	Auth?
(by id) / getPending / hasReviewed / (bare)	GET	Reads.	None
getByCompany	GET	Supports an onlyPublished=1 flag, read via the to_bool() helper (helpers.php) since query-string values arrive as strings, not real booleans.	None
getAverageRating	GET	AVG(note_globale) over published reviews only, rounded to 1 decimal — null if there are no	None

		published reviews yet.	
(create)	POST	Inserts a new review; statut force-set to 'en_attente'.	candidat
approve / reject	PUT	Moderation.	admin — checked once for the whole PUT block
(delete)	DELETE	Removes a review.	admin

DONE WELL Unlike most other files, this one properly gates every write action (candidat to create, admin to approve/reject/delete) — a good example to point to if asked to show a file that does security right.

3.11 categories.php and competences.php — Reference Lists

Both manage a simple {id, nom} lookup table (job categories, and the master skills list respectively) with plain CRUD, properly admin-gated for every write. competences.php adds one extra read-side action, findOrCreateByName — the canonical version of the “find or create” logic that offres.php, candidats.php, and alertes.php each separately re-implement inline rather than calling this shared endpoint (a real, if minor, code-duplication example worth mentioning if asked about refactoring opportunities).

3.12 utilisateurs.php — Accounts (the most nuanced auth logic)

Manages the base utilisateurs record. shape_user() deliberately never includes password_hash in any response — not because the SQL excludes it, but because the shaping function simply never copies that field into its output object.

Table 15 — api/utilisateurs.php — actions

Action	Method	What it does	Auth?
(by id) / getByEmail	GET	Individual lookups.	None
(bare)	GET	Every user account.	admin
markNotificationsSeen	PUT	Updates the “last seen” timestamp.	None

setStatus	PUT	Blocks/unblocks an account (statut).	admin
(update)	PUT	Only the email field is editable this way.	admin
(delete)	DELETE	Deletes an account — with a real ownership check (see below).	any logged-in user

api/utilisateurs.php — DELETE (the most careful check in the whole API)

```
$user = require_role(); // any logged-in user may attempt this
if ($user['role'] !== 'admin' && (int)$id !== (int)$user['userId']) {
    respond_error('Accès refusé.', 403);
}
$pdo->prepare('DELETE FROM utilisateurs WHERE id = ?')->execute([$id]);
if ((int)$id === (int)$user['userId']) { $_SESSION = []; session_destroy(); }
```

An admin can delete any account; a non-admin can only delete their own — enforced by comparing the target id against the logged-in session's own userId, not just checking “are you logged in.” This is the most sophisticated authorization check anywhere in the project, and it's also why `parametres.html`'s “delete my account” button (Part 4.2) logs you out automatically afterward: the code above destroys the session itself the moment you delete your own account.

3.13 upload.php — File Uploads (the most carefully secured file)

Handles multipart file uploads (CVs, photos, logos) and never touches the database at all — it only returns a path string, which the front-end then saves onto a candidate or company record via a normal PUT request afterward.

api/upload.php (condensed)

```
$user = require_role(['candidat', 'recruteur']); // checked before anything else runs

$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $file['tmp_name']); // sniffs the ACTUAL file bytes

$allowed = $kind === 'cv'
    ? ['application/pdf' => 'pdf']
    : ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp', 'image/gif' => 'gif'];

if (!isset($allowed[$mime])) respond_error('Type de fichier non autorisé.', 415);

$filename = bin2hex(random_bytes(8)) . '.' . $allowed[$mime]; // random, not client-chosen
$dir = __DIR__ . '/../uploads/$subdir/{$_SESSION['userId']}"; // path from the SESSION, not the request
```

Four separate security decisions worth being able to explain individually, since this is genuinely the best-written file in the API from a security standpoint:

- The uploaded file's real content is inspected (`finfo_file`) to determine its true MIME type — a client can rename any file to `photo.jpg`, but it can't change what the file's actual bytes say it is. This is

meaningfully stronger than trusting the file extension or the browser-supplied Content-Type header, both of which are just client-provided text.

- The stored filename is cryptographically random (`random_bytes`), not the original filename — this avoids collisions, overwrites, and path-based attacks, while the original filename is still separately returned to the client for display purposes only.
- The storage folder is built from the logged-in session's own `userId`, never from anything the client sends — so there's no request parameter to tamper with in order to write into someone else's folder.
- A file-size cap (5 MB for a CV, 3 MB for an image) is enforced before anything is saved.

Part 4 — Page by Page, In the Order a Real User Sees Them

Every page in the project follows the same loading order in its `<body>`: Bootstrap, then `db.js`, `notifications.js`, `auth.js`, `components.js`, `main.js` — all covered in Part 2 — then (sometimes) `charts.js` or `quiz-data.js`, then the page's own script from `assets/js/pages/`, loaded last. From here on, whenever a walkthrough says “calls `Jobs.getPublished()`” or “calls `Auth.guardPage()`”, that function was already explained in full in Part 2 — this part focuses only on what's specific to each page.

TWO RECURRING SHORTHANDS Two small conventions used throughout Part 4: “(async)” next to a listener means it's an async function, so anything it awaits pauses only that handler, not the page. And “render()-and-rewire” refers to a pattern used on almost every page with buttons in a list or table: after any action that changes data, the page re-fetches and completely rebuilds the relevant HTML from scratch (rather than patching the DOM by hand), and re-attaches every event listener on the fresh elements — because the old DOM nodes (and their listeners) were just destroyed. You'll see “render() again” at the end of nearly every click handler in this part.

4.1 Public Pages — Before You Log In

Our entry point. This is what a visitor sees before ever creating an account, and it's the natural place to start reading the project, since every later page builds on ideas introduced here.

index.html — the Homepage

Purpose: the landing page — a hero search bar, live counts of jobs/companies/candidates, and a preview of the most recently published jobs.

What happens when the page loads (`assets/js/pages/index.js`):

15. `await NeoComponents.renderHeader({rootPrefix:"", active:'home'})` — builds the navbar; `NeoComponents.renderFooter(...)` right after, not awaited.
16. Two functions are defined — `renderFeatured()` and `renderStats()` — and then both are called at the very end without `await`, so they run independently, in the background, at the same time.
17. `renderFeatured()` awaits `Promise.all([Jobs.getPublished(), Companies.getAll()])`, sorts the jobs by date (newest first), then sorts a second time so that jobs from “featured” (premium-plan) companies float

to the top — a classic double-sort trick — keeps the top 6, and renders each with `NeoComponents.jobCard()`.

18. `renderStats()` awaits all three lists in parallel and just writes their `.length` into the three stat counters.

The one interactive element is the hero search form: on submit, it reads the title/city inputs and redirects to `offres.html?q=...&ville=...` — it doesn't search anything itself, it just hands the query to the next page.

offres.html — Search and Filter Jobs

Purpose: the main job board. Filter by keyword, city, category, skill, and contract type; sort; page through results; favorite a job; compare up to 3 side by side; save the current filters as an alert (candidat only).

What happens when the page loads (`assets/js/pages/offres.js`):

19. `await Auth.current()` to find out who (if anyone) is logged in; if it's a candidat, `await Candidates.getByUserId(...)` too.
20. `await Jobs.getPublished()` — fetched once, into `allJobs` — plus `Companies.getAll()`, `Categories.getAll()`, `Competences.getAll()`, and (if a candidat) `Favorites.getByCandidate()`. Everything from here on is filtered/sorted/paged entirely in the browser, in memory — no more network calls happen just from typing in the search box.
21. `prefillFromUrl()` reads `?q=?ville=?categorie=` from the address bar (this is what `index.html`'s search redirect, and the salary simulator's “see matching offers” link, both rely on) and fills the filter inputs before the first render.
22. `render()` is called: computes `getFilteredJobs()` (a pure, synchronous function reading every filter input's current value), slices out the current page (6 per page), and rebuilds the job grid.

Key interactions, each ending in a call back to `render()`:

- Any filter input change resets to page 1 and re-renders — purely client-side, instant.
- The heart-shaped favorite button (async): `await Favorites.toggle(candidateId, jobId)`, toast, `render()`.
- Compare checkboxes: capped at 3 via a JavaScript Set; ticking a 4th reverts the checkbox and shows a warning toast. “Compare” opens a modal building a side-by-side table.
- “Créer une alerte” (candidat only): opens a modal pre-filled with a summary of the current filters; confirming calls `await JobAlerts.create({...})`.

offre-detail.html — One Job's Full Page

Purpose: the full description of one job, the apply flow, and a short list of similar offers.

23. Reads `?id=` from the URL, awaits `Jobs.getById(id)`, `Auth.current()`, and — if a candidat — `Candidates.getByUserId(...)`.
24. A view-counting block runs: it checks `sessionStorage` (not `localStorage` — deliberately scoped to this browser tab/session only) for whether this job id was already counted this visit; if not, it fires `Jobs.recordView(job.id)` without awaiting it (fire-and-forget — the page doesn't need to wait on a view-count increment) and remembers it so a page refresh doesn't double-count.
25. `render()` builds the whole page, calling two async helpers inline: `ctaBlock()` (which returns different HTML depending on state — guest, wrong role, already-applied via `await Applications.hasApplied(...)`, or the “Postuler” button) and `similarJobsHtml()` (fetches published jobs in the same category, excluding this one).

SIMULATED VS. REAL: THE APPLY MODAL'S CV FIELD The apply modal's CV field only captures the selected file's name — no actual file content is uploaded here. The form-text note in the HTML says so directly. Compare this to `candidat/profil.html`'s photo/CV fields (Part 4.2), which do perform a real upload via `Uploads.upload()` — a useful contrast if you're asked to point out something that's simulated versus something that's real.

Submitting the apply form (async): validates the cover-letter text isn't empty, awaits `Applications.create({jobId, candidateId, lettreMotivation, cvName})`, then calls `render()` again so the button immediately reflects “already applied.”

enterprise.html — A Company's Public Page

Purpose: company profile, an optional embedded video, a photo gallery with a lightbox, published employee reviews, and (for a candidat who hasn't already reviewed this company) a “leave a review” flow.

26. `await Companies.getById(id)`; if not found, shows an empty state and stops entirely.
27. `renderPage()` runs `Promise.all([Jobs.getByCompany(...), Reviews.getByCompany(id,{onlyPublished:true}), Reviews.getAverageRating(id)])` together, plus (only if a candidat) `await Reviews.hasReviewed(candidateId, companyId)` to decide whether to show the “write a review” button at all.

Submitting a review (async) reads the four star ratings — generated dynamically each time the modal opens via `NeoUI.starInputHtml/wireStarInput` (Part 2.5), not present in the static HTML at all — then awaits `Reviews.create({...})`. The success toast explicitly tells the user their review will appear “après modération” (after moderation) — because it's fetched with `onlyPublished:true`, a newly submitted review never shows up immediately, by design, until an admin approves it (Part 4.4).

simulateur-salaire.html — Salary Estimator

Purpose: a free tool estimating a salary range from category, city, and experience level — computed live from real published listings, not a hardcoded table.

assets/js/pages/simulateur-salaire.js (condensed)

```
async function estimateSalary(categoryId, ville, experience){
  const published = await Jobs.getPublished();
  const jobsInCat = published.filter(j => String(j.categoryId) === String(categoryId));
  const base = jobsInCat.length
    ? { min: average(jobsInCat, 'salaireMin'), max: average(jobsInCat, 'salaireMax') }
    : FALLBACK_RANGE;
  return {
    min: roundTo100(base.min * EXPERIENCE_MULTIPLIER[experience] * cityMult),
    max: roundTo100(base.max * EXPERIENCE_MULTIPLIER[experience] * cityMult),
    sampleSize: jobsInCat.length,
  };
}
```

On submit, this is awaited once and the result panel is filled with `NeoUI.formatMoney(...)` for both bounds, plus a sentence disclosing the sample size (“based on N published offers...”) so the estimate never pretends to be more authoritative than it is.

tarifs.html — Pricing

Purpose: Free vs. Premium plan comparison for recruiters. `render()` branches on `Auth.current()`'s result: not logged in → both buttons become “create an account” links; logged in but not a recruteur → both disabled; logged in as recruteur → `await Companies.getByUserId(...)` and `await Companies.activeJobCount(...)` to show either “Plan actuel” (already premium) or a live “N/6 active jobs used” message, with the Premium button redirecting to `recruteur/paiement.html`. No purchase happens on this page at all.

contact.html, a-propos.html, faq.html, 404.html — the Static Pages

`contact.html` has a real page script (`contact.js`): it pre-fills the name/email fields if someone's logged in, but its submit handler is explicitly a mock — it shows a success toast and resets the form without ever calling an API. The header comment states this outright: “mock submit, no backend yet.”

`faq.html` and `404.html` have no page script at all — just an inline `<script>NeoComponents.renderHeader(...); NeoComponents.renderFooter(...);</script>` and otherwise fully static content (an accordion of questions; a “page not found” message).

A LIKELY REAL BUG — MISSING AWAIT `a-propos.html` also has no dedicated page script — its inline script calls `Jobs.getPublished().length`, `Companies.getAll().length`, etc. directly, without `await`: `document.getElementById('stat-jobs').textContent = Jobs.getPublished().length;` Since these functions return Promises (see Part 2.1), `.length` on the result of a Promise is undefined —

meaning this page's four stat tiles very likely display “undefined” instead of a number. This is the single most useful concrete bug to bring up if you're asked to demonstrate you understand why await matters: this is exactly the mistake section 1.7 warns about, sitting live in the project. Worth verifying in the browser before you cite it, but the code itself is unambiguous.

register.html and login.html — Creating and Entering an Account

Both start identically: `await Auth.isLoggedIn()` — if already logged in, redirect straight to `Auth.spaceHome((await Auth.current()).role)` and stop, so a logged-in user can never land back on these forms by accident.

`register.html`: clicking a role card (`candidat/recruteur`) toggles which extra fields show (`prénom/nom` vs. `company name/sector`). Submitting validates client-side first (password ≥ 6 characters, confirmation matches, required fields non-empty for the chosen role) before ever calling `await Auth.register(payload)` — only network-validated after every client-side check passes. On success: toast, `NeoUI.confetti()`, then a 900ms `setTimeout` before redirecting, specifically so the user sees the celebration before the page navigates away.

`login.html` additionally renders a clickable list of five demo accounts (`DEMO_ACCOUNTS`, hardcoded directly in `login.js`) that just copy their email/password into the form fields without submitting — a grading/demo convenience. The file's own comment is explicit that these plaintext passwords exist only in this front-end file, never on the server, which only ever stores the bcrypt hash.

candidat-profil.html (project root) — a Candidate's Public CV

Not to be confused with `candidat/profil.html` (Part 4.2) — this is the read-only, public version, reached via `?id=`, viewable by anyone with the link (typically a recruiter). Fetches `await Candidates.getByid(id)` and `await Users.getByid(candidate.userId)`, then renders one linear, non-interactive page: avatar, bio, skill pills (verified ones marked with a checkmark), an optional embedded video, and contact details. The “print/download” button simply calls the browser's native `window.print()` — navigation and the video are both marked with a `.no-print` CSS class so the printed result looks like a clean CV, not a screenshot of the website.

4.2 The Candidat Journey

Every page below starts with the exact same first line — the “page guard” idiom from section 2.2 — so we'll state it once here rather than repeating it five times: `const session = await Auth.guardPage('candidat', './'); if (!session) return; then await`

`NeoComponents.renderHeader({rootPrefix:'../'})` and `NeoComponents.renderSidebar({role:'candidat', active:'...'})`.

candidat/dashboard.html — the Candidat's Home Screen

Purpose: a one-glance summary — application stats, recent applications, personalized job recommendations, a profile-completion checklist, and earned badges.

28. `await` `Candidates.getByUserId(session.userId)`, `then` `await` `Promise.all([Applications.getByCandidate(...), Favorites.getByCandidate(...)])` run together since neither depends on the other.
29. The 4 stat tiles are filled by simply filtering the two arrays already in memory — no further network calls.
30. The 5 most recent applications become table rows via `await Promise.all(recent.map(async app => { const job = await Jobs.getById(...); const company = await Companies.getById(job.companyId); return `<tr>...`; })))` — the “fetch related data for a whole list, in parallel” pattern from section 1.8, applied to table rows this time instead of notifications.
31. Recommended jobs: awaits every published job and every company, computes `NeoUI.matchScore(candidate.skills, job.competences)` for each job not already applied to, keeps only positive scores, sorts descending, and shows the top 3.
32. Profile completion and badges are both pure, synchronous logic over the already-fetched candidate/applications data — rendered via `NeoUI.completionWidgetHtml()` (section 2.5).

STRUCTURE NOTE This page has no separate `render()` function the way most other pages do — it's one long, flat, linear sequence of awaited steps from top to bottom, since nothing on this page is ever re-rendered after a user action (it's a read-only summary).

candidat/profil.html — Edit My Profile (+ the Skill Quiz)

Purpose: edit personal info and bio, upload a real photo and CV, manage the skills list, and take skill-verification quizzes.

This page keeps three separate “draft” variables — `skills`, `pendingPhoto`, `pendingCv` — distinct from the candidate object fetched at load, so edits don't touch the server until the form is actually submitted. Photo and CV uploads are the exception: they upload immediately on file selection (real uploads, via `Uploads.upload(file, 'photo'/'cv')` from section 2.1), only their association to the profile is deferred to the final Save.

assets/js/pages/candidat-profil.js

```
document.getElementById('p-photo').addEventListener('change', async (e) => {
  const file = e.target.files && e.target.files[0];
  if (!file) return;
  try {
    const uploaded = await Uploads.upload(file, 'photo');
    pendingPhoto = uploaded.path;
    renderAvatar();
  } catch (err) { NeoUI.toast(err.message, 'danger'); }
});
```

Adding a skill (Enter key in the skill input) awaits `Competences.findOrCreateByName(raw)` — never inserted client-side without checking the server first, so two candidates typing “react” and “React” both end up pointing at the same canonical skill.

The quiz, step by step:

33. Clicking “Vérifier” next to a skill calls `openQuiz(skillName)`, which looks up `NeoQuizData[skillName]` (quiz-data.js, section 2.7) and renders 3 multiple-choice questions.
34. Submitting the quiz is entirely client-side grading: it counts correct answers against `passed = correct >= Math.ceil(questions.length * 2 / 3)` — no server call is made just to grade it.
35. Only if `passed`: `await Candidates.verifySkill(candidate.id, activeQuizSkill)` — this is the one part that IS persisted server-side (see `api/candidats.php`, section 3.6) — followed by `NeoUI.confetti()` and, after a 1.6-second pause, closing the modal and re-rendering the skill pills with their new checkmark.

Submitting the whole form (async): `await Candidates.update(candidate.id, {..., skills, cvName: pendingCv.name, cvPath: pendingCv.path, photo: pendingPhoto})` — this single call is where all three draft variables finally get committed together.

candidat/candidatures.html — Tracking My Applications

Purpose: every application's current pipeline status, an interview-slot picker when one's been proposed, a per-application message thread, and the ability to withdraw.

The busiest row-building logic in the whole `candidat` side: for every application, `render()` awaits three things in sequence — the job, then the company, then `Interviews.getByApplication(app.id)` — with every row's three-step chain running concurrently across the whole list via the same `Promise.all(map(async...))` pattern used everywhere else.

- “Choisir un créneau” opens `openSlotPicker(interviewId)`: one button per proposed slot, each awaiting `Interviews.selectSlot(interviewId, slot)` on click, then hiding the modal and re-rendering the whole list.
- The message button opens a per-application thread: `renderMessages()` awaits `Messages.getByApplication(...)` and hands it to `NeoUI.messageThreadHtml(messages, 'candidat')` (section 2.5), which aligns the candidate's own messages to the right regardless of who's viewing.

- “Retirer” (withdraw) awaits `NeoUI.confirmDialog(...)` first (section 1.8/2.5) — only on confirmation does it await `Applications.remove(id)`.

candidat/favoris.html — Saved Jobs

Purpose: the candidate's bookmarked jobs, with a one-click way to un-favorite. `render()` awaits `Favorites.getByCandidate(...)`, maps each favorite to a `Jobs.getById(f.jobId)` promise, resolves them all via `Promise.all(...)`, and filters out any null results with `.filter(Boolean)` — protecting against a favorited job that was since deleted. Every card here is rendered with `isFavorite: true` fixed (since, by definition, everything on this page already is one), so the remove button's toast always says “removed”, unlike `offres.html`'s equivalent button, which has to check the toggle's actual return value since it could go either way there.

candidat/alertes.html — My Saved Search Alerts

Purpose: every alert the candidate created from `offres.html`'s filter panel, each showing how many currently-published jobs match it right now.

`render()` awaits `JobAlerts.getByCandidate(...)`, then, for every alert, awaits `JobAlerts.getMatchingJobs(alert)` (section 2.1) to compute a live match count — meaning the count naturally grows over time as new matching jobs get published, since it's recomputed at render time rather than stored on the alert record itself. Deleting an alert follows the same `confirmDialog`-then-`remove`-then-`render()` shape used throughout the site.

parametres.html — Account Settings (Shared by Every Role)

Purpose: theme toggle, a (mostly placeholder) language selector, password change, and account deletion. The one page that guards with `Auth.guardPage(null, '')` — passing null for the role means any logged-in user, of any role, is allowed, which is why `NeoComponents.renderSidebar({role: session.role, ...})` reads the role dynamically here instead of hardcoding it.

- Theme buttons call `NeoTheme.set('dark'/'light')` directly (section 2.5) — not the raw `localStorage` read every page's `<head>` uses to avoid a flash of the wrong theme before CSS loads; that's a separate, earlier mechanism.
- Password change (async): validates the two new-password fields match client-side, then awaits `Auth.changePassword(current, next)` (section 2.2), which itself re-verifies the current password server-side before allowing the change.
- “Delete my account” (async): awaits `NeoUI.confirmDialog(...)`, then, if confirmed, awaits `Users.remove(session.userId)` — the API call whose careful ownership check was walked through in section 3.12 — and hard-redirects to `index.html`.

4.3 The Recruteur Journey

Same guard idiom as 4.2, but with 'recruteur' as the required role, and every page fetches `await Companies.getByUserId(session.userId)` early on to know “my company.”

recruteur/dashboard.html — the Recruiter's Home Screen

Purpose: hiring activity at a glance — job counts by status, an applications-over-time chart, a profile-completion nudge, badges, and the 5 most recent applications.

36. `await Companies.getByUserId(...)`, then plan-dependent branching: premium shows a violet badge; free awaits `Companies.activeJobCount(...)` to show “*Gratuit · N/6 offres.*”
37. `await Jobs.getByCompany(company.id) → jobIds`, then `await Applications.getAll()` filtered client-side down to `jobIds` — note this file has no “applications for my company” endpoint, so it deliberately pulls everything and filters in JavaScript.
38. The 5 most recent rows are built the same `Promise.all(map(async...))` way as the candidat dashboard.
39. `renderApplicationsChart(applications)` — a plain function defined outside the IIFE, called from within it — buckets applications into the last 8 weeks and calls `NeoCharts.barChart('applications-chart', ...)` (section 2.6). It anchors the 8-week window on the most recent application's own date rather than today's real date, specifically so the chart never looks empty against older seed/demo data.

recruteur/poster-offre.html — Publish (or Edit) a Job

Purpose: create a new job posting, or — via `?id=` — edit an existing one owned by this company.

- If `?id=` is present, `await Jobs.getById(editId)` is fetched, and `isEdit` is only true if `existingJob.companyId === company.id` — an ownership check preventing editing a job that isn't yours, even by guessing another job's id in the URL.
- If not editing, `await Companies.hasReachedFreeLimit(company.id)` (section 2.1) decides whether to show the form at all or a “you've hit the free plan limit” panel instead — note editing an existing job never counts against this limit, only creating a new one does.
- Skills use the identical Enter-to-add / `findOrCreateByName` pattern as `candidat/profil.html`'s skill list.
- Submitting (async): if editing, awaits `Jobs.update(id, {...payload, statut:'en_attente'})` — note an edit always resets the job back to pending moderation, requiring re-approval; if creating, awaits `Jobs.create({companyId, ...payload})`. Either way, redirects to `mes-offres.html` afterward.

recruteur/mes-offres.html — My Job Postings

Purpose: every job this company has posted, with status, application count, and per-job actions.

`render()` awaits `Jobs.getByCompany(...)`, then for each job awaits `Applications.getByJob(job.id)` to build an application-count pill that links straight into the Kanban board pre-filtered to that job (`candidatures-recues.html?offre_id=...`) — this is exactly the URL parameter the Kanban page's job-filter dropdown reads on load. The stats modal (`openStats`) awaits the job and its applications again, computes a view→application conversion rate, and draws a small funnel chart using `NeoUI.APP_PIPELINE_ORDER` as the label set — the same pipeline-stage order that drives the Kanban board's own column order.

recruteur/candidatures-recues.html — the Kanban Pipeline (Deep Dive)

This is the single most complex page in the entire project, and the one most worth being able to explain in real detail during your defense — it's the feature that turns NeoJob from “a job board” into “a recruitment tool.” Every application for every job this company posted is shown as a draggable card, arranged into columns by pipeline stage.

Setting the stage

40. After the usual guard + header/sidebar + await `Companies.getById(...)`, the page awaits `Jobs.getByCompany(company.id)` once, up front, building `myJobIds` and a `myJobById` lookup Map — this is the same “resolve it once, look it up many times” idea from section 2.3's `companyBadgeHtml` discussion, applied here to jobs instead of companies.
41. Two module-level Maps — `candidateById` and `interviewByAppId` — are declared but left empty at this point. The code comment explains why directly: they get “populated fresh at the top of every `render()`, so the synchronous card/column HTML builder functions don't need to be async themselves.”

The render cycle

assets/js/pages/recruteur-candidatures.js (condensed/annotated)

```
async function render(){
  if (!myJobs.length) { showEmpty(); return; }
  const apps = await filteredApps(); // Applications.getAll(), filtered to myJobIds
  const candidateIds = [...new Set(apps.map(a => a.candidateId))];

  const [candidates, interviews] = await Promise.all([
    Promise.all(candidateIds.map(id => Candidates.getById(id))),
    Promise.all(apps.map(a => Interviews.getByApplication(a.id))),
  ]);
  candidateById = new Map(candidates.map(c => [c.id, c]));
  interviewByAppId = new Map(apps.map((a,i) => [a.id, interviews[i]]));

  boardEl.innerHTML = APP_PIPELINE_ORDER
    .map(status => columnHtml(status, apps.filter(a => a.statut === status)))
    .join('');
  wireBoard();
}
```

Two `Promise.all()`s nested inside one outer `Promise.all()` — every candidate's data and every application's interview data are all fetched at the same time, none waiting on any other, and only

once that entire batch is back do the two lookup Maps get rebuilt and the (now synchronous) `columnHtml()`/`cardHtml()` functions run over them. `cardHtml(app)` itself never awaits anything — it just reads `candidateById.get(app.candidateId)` and `interviewByAppId.get(app.id)`, both guaranteed to already be populated by this point.

The drag-and-drop mechanics, precisely

`wireBoard()` is called at the end of every single `render()`, because the DOM it's attaching listeners to was just entirely replaced — old listeners on the old (now-discarded) nodes are simply gone.

assets/js/pages/recruteur-candidatures.js

```
card.addEventListener('dragstart', () => { card.classList.add('dragging'); });
card.addEventListener('dragend', () => card.classList.remove('dragging'));
...
column.addEventListener('dragover', (e) => { e.preventDefault(); col.classList.add('drag-over'); });
column.addEventListener('drop', (e) => {
  e.preventDefault();
  col.classList.remove('drag-over');
  const dragging = boardEl.querySelector('.kanban-card.dragging');
  if (!dragging) return;
  moveApplication(Number(dragging.dataset.appId), col.dataset.status);
});
```

Walking through exactly what happens when a recruiter drags a card, step by step:

42. Picking the card up fires `dragstart`, which adds a `.dragging` CSS class to that one card — and that's the entire mechanism used to track “which card is being dragged.” Note this page does not use the browser's `dataTransfer` API at all, which is the more “standard” way to pass data during a drag — instead, the dragged card's identity is found later purely by querying the DOM for whichever element still carries that `.dragging` class.
43. Dragging over a column fires `dragover` repeatedly; `e.preventDefault()` must be called here — this is a real, easy-to-miss browser requirement, since drop targets reject a drop by default unless `dragover` explicitly allows it.
44. Releasing the mouse over a column fires `drop`: it finds the currently-`.dragging` card via `boardEl.querySelector('.kanban-card.dragging')`, reads that card's `data-app-id` and the column's own `data-status` attribute, and calls `moveApplication(appId, newStatus)`.
45. `dragend` fires regardless of whether the drop succeeded, cleaning up the `.dragging` class either way.

assets/js/pages/recruteur-candidatures.js

```
async function moveApplication(appId, newStatus){
  const app = await Applications.getById(appId);
  if (app.statut === newStatus) return; // no-op, nothing actually changed
  await Applications.updateStatus(appId, newStatus); // <- the real network write
  if (newStatus === 'embauchee') NeoUI.confetti();
  NeoUI.toast(`Candidature déplacée vers « ${APP_STATUS[newStatus].label} »`, 'success');
  render(); // rebuild the whole board from fresh data
}
```

This is the exact function that turns a drag-and-drop gesture into a persisted change: it re-fetches the application first (a staleness check), then awaits `Applications.updateStatus(appId, newStatus)` — a PUT request to `api/candidatures.php`'s `updateStatus` action (section 3.7) — and only after that server round-trip completes does it call `render()` again, rebuilding the entire board from the database's new, authoritative state rather than just moving the one card visually and hoping the server agrees. The same `moveApplication()` is also what runs when a recruiter uses the card's “:” dropdown menu instead of dragging — drag-and-drop and the dropdown are two different user interactions that both funnel into this one function.

The detail modal

Clicking a card (anywhere except its dropdown or schedule button) calls `openDetail(appId)`: fetches the application, job, and candidate in sequence, fills the candidate header, meta pills, cover letter, CV filename (or a “no CV attached (simulation)” placeholder — this project never actually stores uploaded CV files from the apply flow, only a filename string, as noted in Part 4.1), the current interview status if one exists, and the message thread via `renderMessages()` (which awaits `Messages.getByApplication(...)` and hands it to `NeoUI.messageThreadHtml(..., 'recruteur')`).

Scheduling an interview from inside this modal (“Planifier un entretien”) does something worth knowing about: instead of calling Bootstrap's normal `.hide()` on the detail modal and waiting for its `hidden.bs.modal` event before opening the interview modal, the code tears the detail modal down synchronously and manually (removing CSS classes and the backdrop element directly) before immediately opening the interview modal. The reason is explained directly in the code's own comment: chaining two Bootstrap modals via the normal hide-then-wait-for-event-then-show sequence is fragile and can leave a stale, invisible backdrop behind if the second modal opens before the first modal's own closing animation has fully finished — forcing an instant, unanimated teardown sidesteps that race condition entirely.

Submitting the interview form (async) reads the mode, location, and up to three datetime-local slot inputs (empty ones filtered out — at least one slot is required, enforced client-side before the request is even sent); if an interview already exists for this application it's updated (which resets any prior candidate slot selection and status back to 'proposee', effectively reopening the choice), otherwise a new one is created.

recruteur/rechercher-candidats.html — the CVthèque

Purpose: a searchable, paginated directory of every candidate on the platform. `await Candidates.getAll()` runs exactly once, up front; every keystroke in the filter inputs afterward is

answered purely from that already-fetched array in memory — the same “fetch once, filter locally” design as `offres.html`.

recruteur/profil.html — Edit My Company

Purpose: edit the company's public profile, logo, photo gallery, and view a read-only billing summary. Follows the identical “draft variable, real upload on selection, commit-on-submit” pattern as `candidat/profil.html` — but with one asymmetry worth knowing: the logo (`pendingLogo`) is staged and only saved when the whole form is submitted, while gallery photos are persisted immediately the moment they're selected (`await Companies.update(id, {photos: company.photos.concat(newPaths)})`), because the gallery has no separate “Save” step of its own in the UI — adding a photo is its own complete action.

recruteur/paiement.html — Simulated Checkout

Purpose: upgrade to Premium via a fake card form or a fake bank-transfer confirmation. Genuinely no real payment processor is involved anywhere — the card number/expiry/CVC fields are auto-formatted as you type (digits-only, spaced every 4, etc.) and validated only for shape (16 digits, MM/AA format), never sent anywhere or checked against a real payment gateway.

assets/js/pages/recruteur-paiement.js (condensed)

```
function processPayment(btn){
  btn.disabled = true;
  btn.innerHTML = '<spinner> Traitement du paiement...';
  setTimeout(async () => {
    const payment = await Payments.create({companyId: company.id, montant: 358.80});
    await Companies.upgradeToPremium(company.id);
    showSuccess(payment);
  }, 1200);
}
```

The 1200ms `setTimeout` exists purely to simulate real processing latency — but notice the two awaited calls inside it are real, genuine API calls that really do persist a payment row and really do flip the company's plan to premium in the actual database; only the “processing” delay and the absence of a real card network are fake, not the resulting application state.

A note on the recruteur journey overall

Notice that every one of these seven pages, from dashboard to Kanban to paiement, shares the exact same skeleton: guard, header, sidebar, fetch “my company,” then page-specific logic. Once you can explain that skeleton once, explaining any individual recruteur page is really just explaining what's different about it — which is exactly how this guide has presented them.

4.4 The Admin Journey

Same guard idiom again, this time with 'admin' as the required role. Admin pages are, structurally, the simplest in the project — almost every one follows `render()` → table of rows → an action button → confirm (for destructive actions) → API call → `render()` again.

admin/dashboard.html — Platform Overview

assets/js/pages/admin-dashboard.js

```
const [users, jobs, companies, payments, categories] = await Promise.all([
  Users.getAll(), Jobs.getAll(), Companies.getAll(), Payments.getAll(), Categories.getAll(),
]);
```

Five completely independent fetches, all fired at the same moment via one `Promise.all` — the cleanest example in the project of “these don't depend on each other, so don't make the admin wait for them one at a time.” Everything else on this page — six stat tiles, a pending-jobs table, a role-breakdown of every user, and a per-category published-jobs chart — is pure in-memory filtering/counting over those five already-fetched arrays. Nothing on this page can be clicked to change anything; it's read-only by design.

admin/moderation.html — Approve or Reject Job Postings

Purpose: the queue every newly-posted or newly-edited job passes through before it's publicly visible (recall from Part 4.3 that both creating and editing a job resets its status to `en_attente`).

One detail worth knowing: the action buttons are relabeled based on context but map to the exact same two underlying calls every time — a pending job shows “Approuver”/“Refuser”, a published job shows a single “Dépublier” button, and a rejected job shows a single “Republier” button, but “Republier” and “Approuver” both just call `await Jobs.approve(id)`, and “Dépublier”/“Refuser” both call `await Jobs.reject(id)` — the UI is relabeling the same two server actions differently depending on the job's current state, not implementing four separate behaviors.

admin/utilisateurs.html — Manage Every Account

Purpose: list every user, filter by role, block/unblock or permanently delete non-admin accounts.

For each user, an async helper `profileFor(user)` resolves the right linked profile — `Candidates.getByUserId` for a candidat, `Companies.getByUserId` for a recruteur, null for an admin — so the table can show a real name and photo/logo instead of just an email address. Admin accounts are the one row type where the block and delete buttons are replaced with a plain “—”: this page's own UI enforces that admins can't be blocked or deleted from here at all (a UI-level restriction,

distinct from — and stricter than — the ownership check inside `api/utilisateurs.php`'s own DELETE handler covered in section 3.12, which would in fact allow it at the API level if called directly).

admin/entreprises.html — Grant or Revoke the Verified Badge

Purpose: the “verified company” checkmark that shows up on a company's public profile and job cards throughout the site. For each company, `render()` awaits `Jobs.getByCompany(c.id)` just to display a job count alongside it. The toggle button re-fetches the company fresh right before flipping its state (`await Companies.getByid(id)` then `await Companies.setVerified(id, !c.verified)`) rather than trusting whatever verified value was baked into the row the last time the table was rendered — a small but deliberate staleness guard.

admin/avis.html — Moderate Company Reviews

Purpose: approve or reject employee reviews before they can appear publicly on `entreprise.html` (Part 4.1).

Opening a review's detail modal (`openReview`) awaits the review, its company, and its author candidate in sequence, and — worth calling out specifically — shows the reviewer's name to the admin with the note “(masqué publiquement)” (hidden from the public) right next to it: the admin can see who wrote a review while moderating it, even though the public-facing version of the review never reveals the author's identity. The two footer buttons in the modal both read data-status off themselves and route to either `Reviews.approve(id)` or `Reviews.reject(id)` depending on which was clicked.

admin/categories.html and admin/competences.html — the Reference-List Editors

Structurally near-identical CRUD pages for the job-categories list and the skills/competences master list, each with the same `resetForm()` / `render()` / edit / delete shape used throughout the admin side. The one detail worth knowing on both: the delete button is disabled outright, with an explanatory tooltip, whenever that category or skill is still in use — `competences.html`'s `usageCount(nom, jobs, candidates)` helper counts how many jobs list a skill and how many candidates list it separately, disabling deletion if either count is above zero. This is a UI-level safeguard against an admin accidentally breaking existing job postings or candidate profiles by deleting a category or skill they're actively relying on.

Part 5 — Things Worth Knowing for Q&A

5.1 Recurring Patterns Cheat-Sheet

If you can recognize and explain each of these five patterns on sight, you can explain essentially any page in the project, even one this guide doesn't spell out line by line.

Table 16 — The five patterns that repeat across almost every page

Pattern	What it looks like	Why it exists
The page guard	<code>const session = await Auth.guardPage(role, prefix); if (!session) return;</code>	Redirects an unauthorized visitor before any data is fetched or any HTML is built for a page they shouldn't see.
render()-and-rewire	An <code>async render()</code> function, called again at the end of nearly every button's click handler.	The DOM was just rebuilt from scratch after any change, so old event listeners (on now-discarded elements) must be re-attached to the new ones — refetching and rebuilding is simpler and more reliably correct than patching the DOM by hand.
<code>Promise.all(list.map(async x => ...))</code>	Building table rows or cards where each one needs its own extra lookup (a job's company, an application's candidate).	Runs every item's lookup concurrently instead of one at a time — the whole list loads roughly as fast as the single slowest lookup, not the sum of all of them.
Stays-sync-on-purpose functions	<code>jobCard()</code> , <code>candidateCard()</code> , <code>JobAlerts.matches()</code> , <code>cardHtml()</code> in the Kanban page.	These run inside tight render loops; the caller resolves any needed data once, up front, and passes it in, so the render function itself never has to await anything.
Manually-built Promises	<code>NeoUI.confirmDialog()</code> , <code>NeoUI.readFileAsDataURL()</code> .	Proof that <code>await</code> isn't only for network calls — a Promise can represent waiting on anything eventual, including a human clicking a button.

5.2 Known Bugs and Gaps — Good, Honest Answers if Asked “What Would You Improve?”

Table 17 — Real issues found while researching this guide

Where	The issue
a-propos.html (inline script)	Calls <code>Jobs.getPublished().length</code> etc. without <code>await</code> — since these return Promises, the four stat tiles likely display “undefined”. Worth verifying live, but the code itself is unambiguous (section 4.1).
assets/js/quiz-data.js	The 'Java' and 'Kotlin' skill-quiz entries contain the exact same three SEO questions as the 'SEO' entry, word for word — a copy-paste content bug, not real Java/Kotlin content (section 2.7).
api/candidats.php	Zero <code>require_role()</code> calls anywhere in the file — every action, including listing every candidate's full profile, is open to anyone (section 3.6).
api/offres.php	Only job creation requires login; approving, rejecting, editing, and deleting a job posting have no auth check at all (section 3.4).
api/entreprises.php	Upgrading a company to premium and toggling its verified badge have no auth check (section 3.5).
api/messages.php	A message's <code>senderRole</code> is trusted from the request body rather than re-derived from the actual logged-in session (section 3.9).
Duplicated logic	The filter-criteria summary shown in <code>offres.html</code> 's alert modal is near-identical to, but separately re-implemented in, <code>candidat-alertes.js</code> — not factored into one shared function (sections 4.1 and 4.2).
competences.php's <code>findOrCreateByName</code>	The canonical version of a “find or create” pattern that <code>offres.php</code> , <code>candidats.php</code> , and <code>alertes.php</code> each separately re-implement inline instead of reusing (section 3.11).

A NOTE ON WHY THIS SECTION EXISTS None of this is presented to embarrass the project — a real, working, full-stack application with a genuine security gap or two is completely normal, and being able to name yours specifically (rather than claiming everything is flawless) reads as far more credible in a defense than pretending nothing could be improved.

5.3 Quick Glossary

Table 18 — Terms used throughout this guide

Term	Plain-English meaning
API	A fixed set of “doors” a program exposes so other programs can ask it to do things, without needing to know how it works inside (section 1.5).
REST	A style of API where each file represents one resource, and the HTTP method (GET/POST/PUT/DELETE) signals what kind of operation is happening.
JSON	Plain text written in a structured, {key: value} shape that both JavaScript and PHP can read and write natively — the format every API response is sent in.
Promise	An object standing in for a value that isn't ready yet, but will either resolve (succeed) or reject (fail) at some point in the future.
async / await	async lets a function use await inside it; await pauses just that function at that line until a Promise finishes, without freezing the rest of the page.
PDO	PHP's built-in, safe toolkit for talking to a database, used via prepared statements to prevent SQL injection.
Prepared statement	A SQL query written with ? placeholders, with real values supplied separately via ->execute([...]) — the database always treats them as data, never as part of the command itself.
Session / cookie	How the server remembers who's logged in across separate requests, via a small id the browser automatically resends with every request to the same site.
Join table	A small table storing only pairs of ids, used to represent a many-to-many relationship (e.g. offre_competences links jobs to their required skills).
IIFE	Immediately Invoked Function Expression — (async () => { ... })(); a small, self-running function, used here

	purely so the word await is legal inside it.
--	--

Closing Note

Every claim in this guide traces back to an actual file in the project — there's no simplification or guessing about what the code does, only about how to explain it clearly. If a question in your defense goes beyond what's written here, the honest and genuinely strong answer is almost always to open the relevant file and read it together — every pattern this guide walks through repeats constantly, so by the time you've read this far, you already have the tools to work out any function you haven't seen explained explicitly.